is to offer a REST API to find the users by their name. The domain model is reproduced in Figure 1. It primarily consists of value objects like *Name*, *PhoneNumber* along with *User* entity and *UserRepository*.

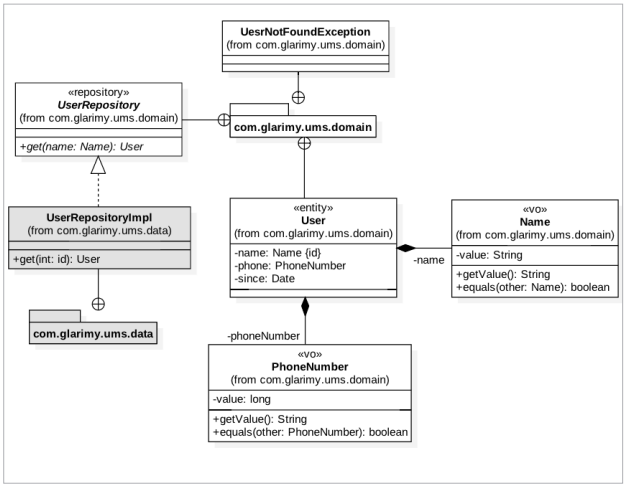

Figure 1: The domain model of *FindService*

Let us begin with the *Name*. Since it is a value object, it must be validated by the time it is created and must remain immutable. The basic structure looks like this:

```
class Name:
  value: str
  def __post_init__(self):
    if self.value is None or len(self.value.strip()) < 8 or
len(self.value.strip()) > 32:
      raise ValueError("Invalid Name")
```

As you can see, the *Name* consists of a *value* attribute of type *str*. As part of the post initialization, the *value* is validated.

Python 3.7 offers *@dataclass* decorator, which offers many features of a data-carrying class out-of-the-box like constructors, comparison operators, etc.

Following is the decorated *Name*:

```
from dataclasses import dataclass

@dataclass
class Name:
  value: str
  def __post_init__(self):
    if self.value is None or len(self.value.strip()) < 8 or
len(self.value.strip()) > 32:
      raise ValueError("Invalid Name")
```

The following code can create an object *Name:*

```
name = Name("Krishna")
```

The value attribute can be read or written as follows:

```
name.value = "Mohan"
print(name.value)
```

Another *Name* object can be compared as easily as follows:

```
other = Name("Mohan)
if name == other: print("same")
```

As you can see, the objects are compared by their values, not by their references. This is all out-of-the-box. We can also make the object immutable, by freezing. Here is the final version of *Name* as a value object:

```
from dataclasses import dataclass

@dataclass(frozen=True)
class Name:
  value: str
  def __post_init__(self):
    if self.value is None or len(self.value.strip()) < 8 or
len(self.value.strip()) > 32:
      raise ValueError("Invalid Name")
```

The *PhoneNumber* also follows a similar approach, as it is also a value object:

```
@dataclass(frozen=True)
class PhoneNumber:
  value: int
  def __post_init__(self):
    if self.value < 9000000000:
      raise ValueError("Invalid Phone Number")
```

The *User* class is an entity, not a value object. In other words, the *User* is not immutable. Here is the structure:

```
from dataclasses import dataclass
import datetime

@dataclass
class User:
  _name: Name
  _phone: PhoneNumber
  _since: datetime.datetime

  def __post_init__(self):
    if self._name is None or self._phone is None:
```